

FAQ

This page is for short answers and quick routing. If you need step-by-step pipeline guidance, start with [usage.md](#).

What is DataSyncX?

DataSyncX is a Python-based ETL framework that standardizes reader/processor/writer pipelines and provides orchestration utilities such as query builders, task ranges, and controlled execution.

For the product overview and documentation map, see [overview.md](#).

Which usage path should I start with?

Start with the [method selection guide](#) in [usage.md](#).

- Choose free-style scripting when you want full control over custom logic.
- Choose direct demo mode when you want the fastest source-to-target starting point.
- Choose demo factory mode when you want repeatable YAML/JSON-driven script generation.
- Choose the custom starter demo when built-in demos are not close to your use case.

When should I use direct demo mode?

Use direct demo mode when an existing built-in source-to-target script is already close to your scenario and you want a fast copy-edit-run workflow.

See [usage.md](#) for the copy flow and [the demo examples section](#) for common split patterns.

When should I use demo factory mode?

Use demo factory mode when you want a config-driven way to generate scripts from YAML or JSON and keep team workflows repeatable.

See [usage.md](#) for the end-to-end flow.

When should I build a custom pipeline?

Use a custom pipeline when you need full control over transforms or want to integrate DataSyncX readers and writers into your own threading or orchestration logic.

For the lightest starting point, see [usage.md](#). For a richer composition pattern, see [the multi-source example](#).

How do I customize a copied demo?

After copying a demo, update the connection settings, SQL, target table details, and any runtime split parameters for your scenario.

If the demo uses time or token splitting, review [Demo examples explained](#) before editing the script heavily.

How do I add credentials?

Create a `.env` file in your project root. Make sure your project `.gitignore` ignores `.env` so credentials are not committed. See the Configuration section in [installation.md](#) for the format and examples.

Why do I see import errors after copying a demo?

Make sure `datasyncx` is installed in your active environment and that you are running the script with that same interpreter.

If setup is incomplete, go back to [installation.md](#). If imports fail only after copying a demo, confirm you copied the script into a project that can import the installed package.

How do I find the right demo?

Demo file names follow the `Source_to_Target.py` naming convention. Use `dsx.list_demos()` to list them and `dsx.copy_demo()` to copy one into your project.

If you are not sure which workflow fits best, start with [the usage decision guide](#).

Do I need a processor?

Not always. Many pipelines only need a reader and writer. Add a processor when you want reusable transformation logic between read and write steps.

See [processors.md](#) for processor-specific guidance.

How do alerts work in demos?

There are two common alert paths:

- Framework alerts during `TaskRun` for runtime reader/processor/writer failures.
- `email_on_exception` for demo-level failures that happen before `TaskRun` starts.

See [the alerting overview in usage.md](#) for when to use each.

Where should I check release changes or upgrade impact?

Use [changelog.md](#) for user-facing changes and [release-notes/index.md](#) for versioned developer notes.